

Machine Learning

Volker Roth

Department of Mathematics & Computer Science
University of Basel

Chapter 11: Neural Encoder-Decoder Models

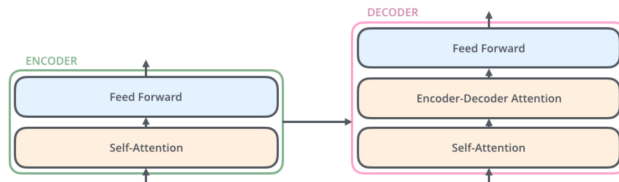


Figure 16.1 in the supplement of K.Murphy: Probabilistic Machine Learning, Advanced Topics. MIT Press, 2023. High level structure of the encoder-decoder transformer architecture. <https://jalammar.github.io/illustrated-transformer/>

Non-linear latent variable models

Latent variable $\mathbf{z} \rightsquigarrow$ Gaussian likelihood with

nonlinearly transformed mean

$$\mu = f(\mathbf{z}, \phi).$$

Prior and likelihood:

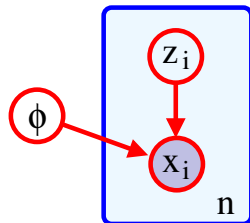
$$p(\mathbf{z}) = N(\mathbf{0}, I)$$

$$p(\mathbf{x}|\mathbf{z}, \phi) = N(\mathbf{f}(\mathbf{z}, \phi), \sigma^2 I).$$

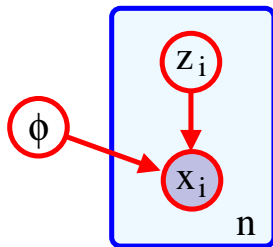
- Given observed $\mathbf{x}$, we want to understand what possible values of the hidden variable $\mathbf{z}$ were responsible for it:

$$p(\mathbf{z}|\mathbf{x}) = \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})}.$$

- No closed form expression available. Cannot evaluate denominator $p(\mathbf{x})$ and so we can't even compute the numerical value of the posterior for a given pair $\mathbf{z}$ and $\mathbf{x}$.



Sampling



- ...but it is easy to generate a new sample $\mathbf{x}^*$ using sampling:
 - ▶ Draw $\mathbf{z}^*$ from the prior $p(\mathbf{z})$, pass this through $f(\mathbf{z}^*, \phi)$
 $\rightsquigarrow$ mean of likelihood $p(\mathbf{x}^*|\mathbf{z}^*)$,
 - ▶ then draw $\mathbf{x}^*$ from this distribution.
- Prior and likelihood are normal distributions $\rightsquigarrow$ sampling is easy.

Evaluating marginal likelihood (evidence)

$$\begin{aligned} p(\mathbf{x}|\phi) &= \int p(\mathbf{x}, \mathbf{z}|\phi) d\mathbf{z} \\ &= \int p(\mathbf{x}|\mathbf{z}, \phi) p(\mathbf{z}) d\mathbf{z} \\ &= \int N(\mathbf{f}[\mathbf{z}, \phi], \sigma^2 I) \cdot N(\mathbf{0}, I) d\mathbf{z}. \end{aligned}$$

No closed form for this integral $\rightsquigarrow$ lower bound (Jensen's inequality):

$$\begin{aligned} \log[p(\mathbf{x}|\phi)] &= \log \left[\int p(\mathbf{x}, \mathbf{z}|\phi) d\mathbf{z} \right] \\ &= \log \left[\int q(\mathbf{z}) \frac{p(\mathbf{x}, \mathbf{z}|\phi)}{q(\mathbf{z})} d\mathbf{z} \right] \\ &\geq \int q(\mathbf{z}) \log \left[\frac{p(\mathbf{x}, \mathbf{z}|\phi)}{q(\mathbf{z})} \right] d\mathbf{z}, \end{aligned}$$

Known as the evidence lower bound ELBO, because $p(\mathbf{x}|\phi)$ is the evidence (= marginal likelihood) in the context of Bayes' rule.

- In practice, the distribution $q(\mathbf{z})$ will have some parameters θ :

$$\text{ELBO}[\theta, \phi] = \int q(\mathbf{z}|\theta) \log \left[\frac{p(\mathbf{x}, \mathbf{z}|\phi)}{q(\mathbf{z}|\theta)} \right] d\mathbf{z}.$$

- To learn the non-linear latent variable model, we'll maximize this quantity as a function of both ϕ and θ .
- We will see: the maximum is obtained (theoretically) if the variational distribution is the true posterior, $q(\mathbf{z}|\theta) = p(\mathbf{z}|\mathbf{x}, \phi)$.
- In practice, we maximize ELBO over some tractable family of distributions $q(\mathbf{z}|\mathbf{x}, \theta)$ to obtain an approximation of the intractable posterior.
- The neural architecture that computes this is the **variational autoencoder**.

Tightness of ELBO

$$\begin{aligned}\text{ELBO}[\theta, \phi] &= \int q(\mathbf{z}|\theta) \log \left[\frac{p(\mathbf{x}, \mathbf{z}|\phi)}{q(\mathbf{z}|\theta)} \right] d\mathbf{z} \\&= \int q(\mathbf{z}|\theta) \log \left[\frac{p(\mathbf{z}|\mathbf{x}, \phi)p(\mathbf{x}|\phi)}{q(\mathbf{z}|\theta)} \right] d\mathbf{z} \\&= \int q(\mathbf{z}|\theta) \log [p(\mathbf{x}|\phi)] d\mathbf{z} + \int q(\mathbf{z}|\theta) \log \left[\frac{p(\mathbf{z}|\mathbf{x}, \phi)}{q(\mathbf{z}|\theta)} \right] d\mathbf{z} \\&= \log[p(\mathbf{x}|\phi)] + \int q(\mathbf{z}|\theta) \log \left[\frac{p(\mathbf{z}|\mathbf{x}, \phi)}{q(\mathbf{z}|\theta)} \right] d\mathbf{z} \\&= \log[p(\mathbf{x}|\phi)] - D_{KL} [q(\mathbf{z}|\theta) \| p(\mathbf{z}|\mathbf{x}, \phi)].\end{aligned}$$

ELBO is the log marginal likelihood minus $D_{KL} [q(\mathbf{z}|\theta) \| p(\mathbf{z}|\mathbf{x}, \phi)]$.
 D_{KL} zero when $q(\mathbf{z}|\theta) = p(\mathbf{z}|\mathbf{x}, \phi) \rightsquigarrow \text{ELBO} = \log[p(\mathbf{x}|\phi)]$.

ELBO as reconstruction loss minus KL to prior

$$\begin{aligned}\text{ELBO}[\theta, \phi] &= \int q(\mathbf{z}|\theta) \log \left[\frac{p(\mathbf{x}, \mathbf{z}|\phi)}{q(\mathbf{z}|\theta)} \right] d\mathbf{z} \\&= \int q(\mathbf{z}|\theta) \log \left[\frac{p(\mathbf{x}|\mathbf{z}, \phi)p(\mathbf{z})}{q(\mathbf{z}|\theta)} \right] d\mathbf{z} \\&= \int q(\mathbf{z}|\theta) \log [p(\mathbf{x}|\mathbf{z}, \phi)] d\mathbf{z} + \int q(\mathbf{z}|\theta) \log \left[\frac{p(\mathbf{z})}{q(\mathbf{z}|\theta)} \right] d\mathbf{z} \\&= \int q(\mathbf{z}|\theta) \log [p(\mathbf{x}|\mathbf{z}, \phi)] d\mathbf{z} - D_{\text{KL}}[q(\mathbf{z}|\theta), p(\mathbf{z})]\end{aligned}$$

- First term measures the average agreement $p(\mathbf{x}|\mathbf{z}, \phi)$ of the hidden variable and the data (reconstruction loss)
- Second one measures the degree to which the auxiliary distribution $q(\mathbf{z}, \theta)$ matches the prior.

The variational approximation

- ELBO is tight when we choose $q(\mathbf{z}|\boldsymbol{\theta}) = p(\mathbf{z}|\mathbf{x}, \phi)$.
- Intractable $\rightsquigarrow$ variational approximation: choose simple parametric form for $q(\mathbf{z}|\boldsymbol{\theta})$, use it as an approximation to the true posterior.
- Choose a normal distribution with parameters $\boldsymbol{\mu}$ and $\Sigma = \sigma^2 I$.
- Optimization $\rightsquigarrow$ find normal distribution closest to true posterior $p(\mathbf{z}|\mathbf{x})$. Corresponds to minimizing the KL divergence.
- True posterior $p(\mathbf{z}|\mathbf{x})$ depends on $\mathbf{x}$
 $\rightsquigarrow$ variational approximation should also depend on $\mathbf{x}$:

$$q(\mathbf{z}|\boldsymbol{\theta}, \mathbf{x}) = N(g_{\boldsymbol{\mu}}[\mathbf{x}|\boldsymbol{\theta}], g_{\sigma^2}[\mathbf{x}|\boldsymbol{\theta}]),$$

where $g[\mathbf{x}, \boldsymbol{\theta}]$ is a neural network with parameters $\boldsymbol{\theta}$.

The variational autoencoder

Recall

$$\text{ELBO}[\theta, \phi] = \int q(\mathbf{z}|\mathbf{x}, \theta) \log [p(\mathbf{x}|\mathbf{z}, \phi)] d\mathbf{z} - D_{KL}[q(\mathbf{z}|\mathbf{x}, \theta), p(\mathbf{z})]$$

Involves an intractable integral, but it is an expectation

↪ approximate with samples:

$$E_{q(\mathbf{z}|\mathbf{x}, \theta)}[\log [p(\mathbf{x}|\mathbf{z}, \phi)]] \approx \frac{1}{N} \sum_{n=1}^N \log [p(\mathbf{x}|\mathbf{z}_n^*, \phi)]$$

where $\mathbf{z}_n^*$ is the n -th sample from $q(\mathbf{z}|\mathbf{x}, \theta)$. Limit: use a single sample:

$$\text{ELBO}[\theta, \phi] \approx \log [p(\mathbf{x}|\mathbf{z}^*, \phi)] - D_{KL}[q(\mathbf{z}|\mathbf{x}, \theta), p(\mathbf{z})]$$

The second term is just the KL divergence between two Gaussians and is available in closed form.

The reparameterization trick

Recall: Want to sample from

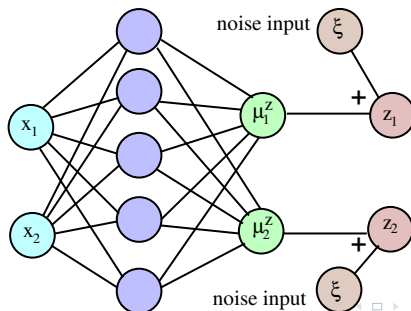
$$q(\mathbf{z}|\boldsymbol{\theta}, \mathbf{x}) = N(g_{\mu}[\mathbf{x}|\boldsymbol{\theta}], g_{\sigma^2}[\mathbf{x}|\boldsymbol{\theta}]),$$

To let PyTorch / Tensorflow perform automatic differentiation via backpropagation, we must avoid the sampling step.

Simple solution: draw a sample $\boldsymbol{\xi} \sim N(0, I)$ and use

$$\mathbf{z}^* = g_{\mu} + g_{\sigma^2}^{1/2} \boldsymbol{\xi}.$$

Now “the gradient can flow through the network”. **Encoder network:**



- Finally, minimize negative expectation of ELBO over $p(\mathbf{x})$:

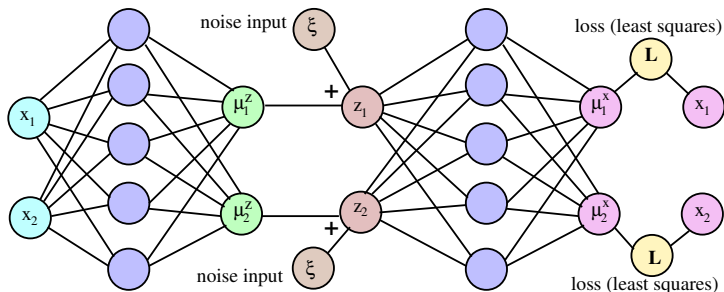
$$\min_{\phi, \theta} -E_{p(\mathbf{x})} E_{q(\mathbf{z}|\mathbf{x}, \theta)} [\log [p(\mathbf{x}|\mathbf{z}, \phi)]] + E_{p(\mathbf{x})} D_{KL}[q(\mathbf{z}|\mathbf{x}, \theta), p(\mathbf{z})]$$

- The first term is approximated as

$$E_{p(\mathbf{x})} E_{q(\mathbf{z}|\mathbf{x}, \theta)} [\log [p(\mathbf{x}|\mathbf{z}, \phi)]] \approx \frac{1}{n} \sum_{i=1}^n \log [p(\mathbf{x}_i|\mathbf{z}_i^*, \phi)].$$

We assume $p(\mathbf{x}_i|\mathbf{z}_i^*, \phi) = \mathcal{N}(f_\phi(\mathbf{z}_i^*), \sigma^2)$,

where f is implemented via a neural net: $\rightsquigarrow$ **Decoder network**

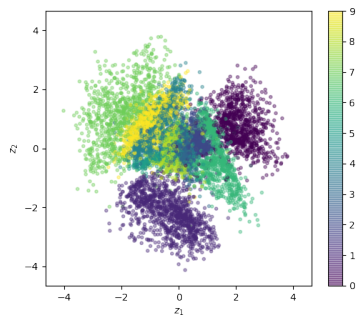
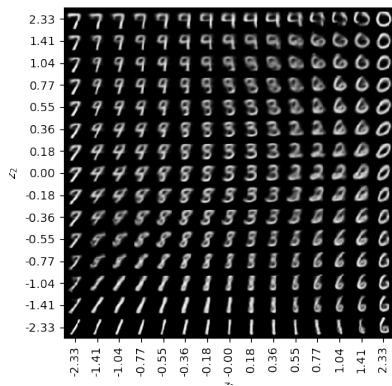


Further Variations

- For maximizing ELBO, we jointly optimize over the parameters of encoder and decoder network.
- When adjusting the decoder, we also change the “true” posterior that we are going to approximate!
- So approximation quality should not be our only goal... need “tuning knob” for steering the model into a desired direction.
- Solution: introduce parameter $\beta > 0$ that controls the relative importance of the two loss terms:

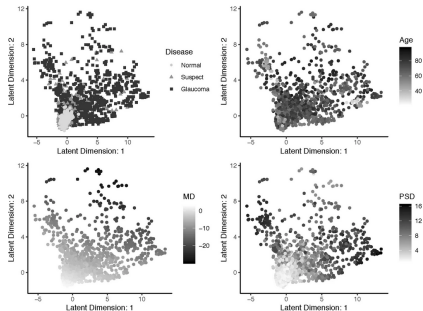
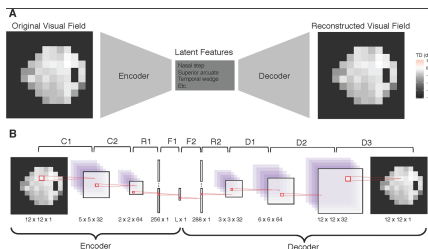
$$\min_{\theta, \phi} \frac{1}{n} \sum_{i=1}^n D_{KL}[q(\mathbf{z}|\mathbf{x}_i, \theta), p(\mathbf{z})] - \beta \frac{1}{n} \sum_{i=1}^n \log [p(\mathbf{x}_i|\mathbf{z}_i^*, \phi)]$$

Applications: MNIST example



Taken from Louis Tiago: A Tutorial on Variational Autoencoders with a Concise Keras Implementation

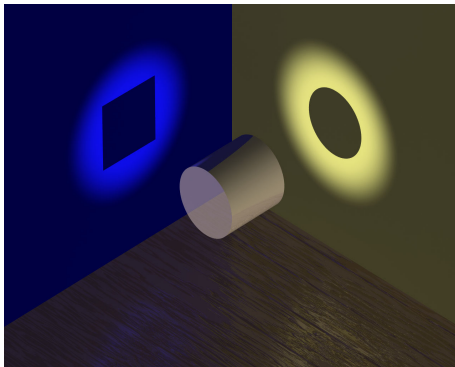
Applications: Medical example



Berchuck, S.I., Mukherjee, S. & Medeiros, F.A. Estimating Rates of Progression and Predicting Future Visual Fields in Glaucoma Using a Deep Variational Autoencoder. Sci Rep 9, 18113 (2019). <https://doi.org/10.1038/s41598-019-54653-6>

Multiple Views: Deep Information Bottleneck

- Consider paired samples from different views.
- What is the dependency structure between the views ?
- Nonlinear model: dependency detected by **deep IB**.



Two-view version: The deep information bottleneck

- So far we argued that since the true posterior $p(\mathbf{z}|\mathbf{x})$ depends on $\mathbf{x}$, the variational approximation should also depend on $\mathbf{x}$.
- Restricted setting: explain posterior **only by external variable** $\tilde{\mathbf{x}}$:

$$q = q(\mathbf{z}|\boldsymbol{\theta}, \tilde{\mathbf{x}}).$$

$$\begin{aligned}\text{ELBO}[\boldsymbol{\theta}, \phi] &= \int q(\mathbf{z}|\tilde{\mathbf{x}}, \boldsymbol{\theta}) \log [p(\mathbf{x}|\mathbf{z}, \phi)] d\mathbf{z} - D_{KL}[q(\mathbf{z}|\tilde{\mathbf{x}}, \boldsymbol{\theta}), p(\mathbf{z})] \\ &= E_{q(\mathbf{z}|\tilde{\mathbf{x}}, \boldsymbol{\theta})} \log [p(\mathbf{x}|\mathbf{z}, \phi)] - D_{KL}[q(\mathbf{z}|\tilde{\mathbf{x}}, \boldsymbol{\theta}), p(\mathbf{z})]\end{aligned}$$

- Connection to IB:

- ▶ Assume (or define) $q(\mathbf{z}|\tilde{\mathbf{x}}, \boldsymbol{\theta}) := p(\mathbf{z}|\tilde{\mathbf{x}}, \boldsymbol{\theta})$
- ▶ Take expectation w.r.t. joint data distribution $p(\tilde{\mathbf{x}}, \mathbf{x})$:

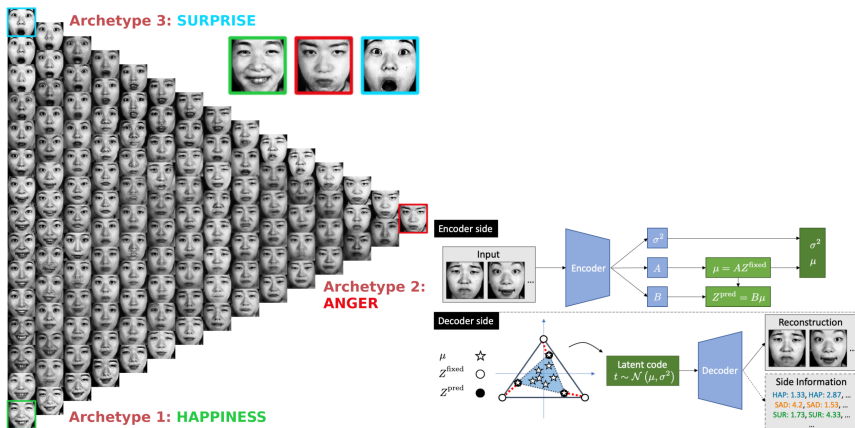
$$E_{p(\tilde{\mathbf{x}}, \mathbf{x})} E_{p(\mathbf{z}|\tilde{\mathbf{x}}, \boldsymbol{\theta})} \log [p(\mathbf{x}|\mathbf{z}, \phi)] - E_{p(\tilde{\mathbf{x}})} D_{KL}[p(\mathbf{z}|\tilde{\mathbf{x}}, \boldsymbol{\theta}), p(\mathbf{z})]$$

- ▶ First term $\leq \mathcal{I}_{\boldsymbol{\theta}, \phi}(\mathbf{z}; \mathbf{x}) + \text{const.}$ Second term $= \mathcal{I}_{\boldsymbol{\theta}}(\tilde{\mathbf{x}}; \mathbf{z})$,

- This defines the deep information bottleneck (with weight β)

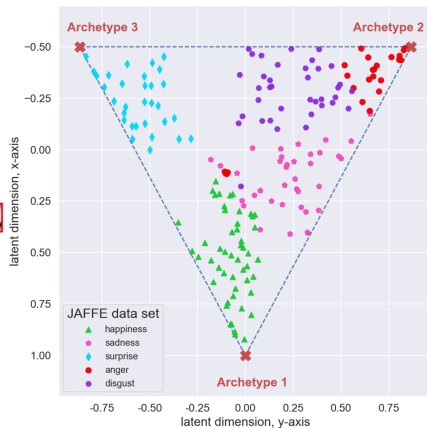
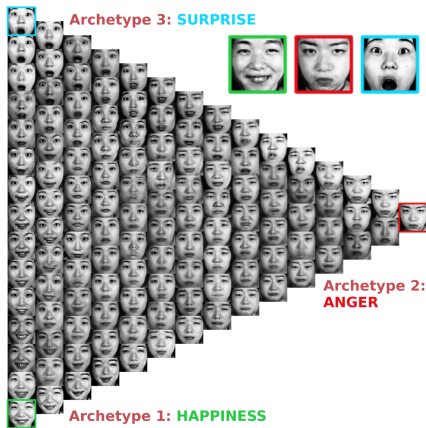
$$\min_{\phi, \boldsymbol{\theta}} \mathcal{I}_{\boldsymbol{\theta}}(\tilde{\mathbf{x}}; \mathbf{z}) - \beta \mathcal{I}_{\boldsymbol{\theta}, \phi}^{\text{low}}(\mathbf{z}; \mathbf{x}), \quad \text{where } \mathcal{I}^{\text{low}} \text{ is a lower bound of } \mathcal{I}.$$

Applications: Face images



Keller et al. 2020: Learning Extremal Representations with Deep Archetypal Analysis

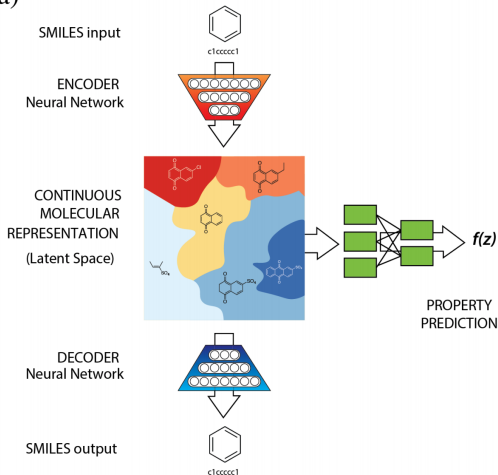
Applications: Face images



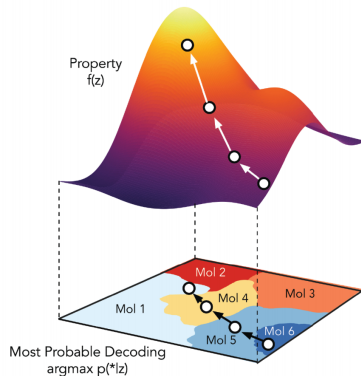
Keller et al. 2020: Learning Extremal Representations with Deep Archetypal Analysis

Applications: Deep Chemical Variational Autoencoders

(a)

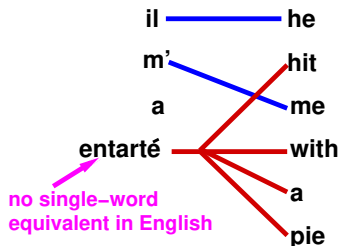


(b)

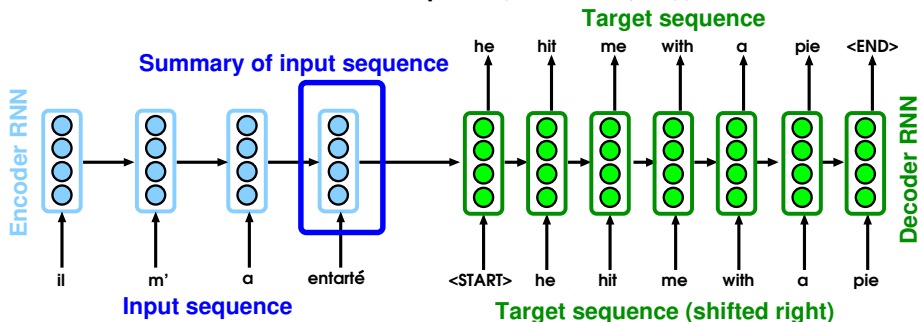


(Gomez-Bombarelli et al., ACS Cent Sci, 2018)

Neural Encoder/Decoder Models for Machine Translation



<https://commons.wikimedia.org/w/index.php?curid=9105527>



Neural Machine Translation (NMT)

- The sequence-to-sequence model is an example of a **Conditional Language Model**:
 - ▶ **Language Model** because the decoder is predicting the next word of the target sentence $\mathbf{y} = \{y_1, y_2, \dots, y_T\}$
 - ▶ **Conditional** because its predictions are also conditioned on the source sentence $\mathbf{x}$.

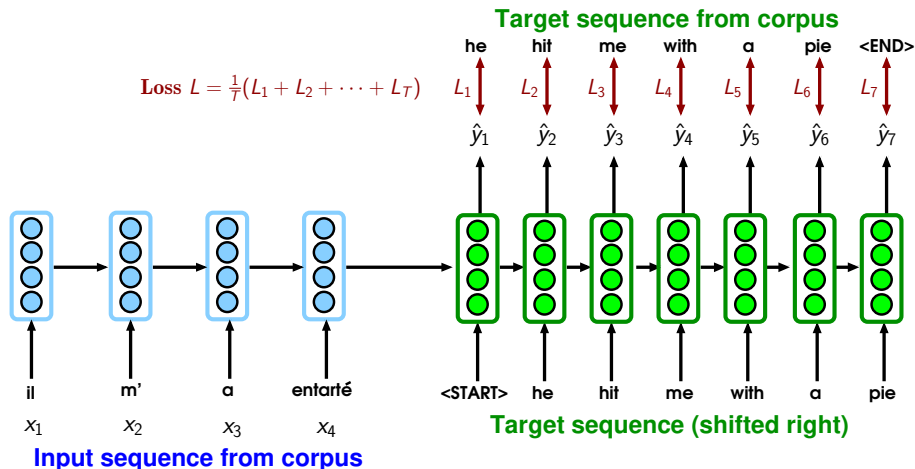
- NMT directly calculates

$$P(\mathbf{y}|\mathbf{x}) = P(y_1|\mathbf{x}) \cdot P(y_2|y_1, \mathbf{x}) \cdots P(y_T|y_{1:(T-1)}, \mathbf{x})$$

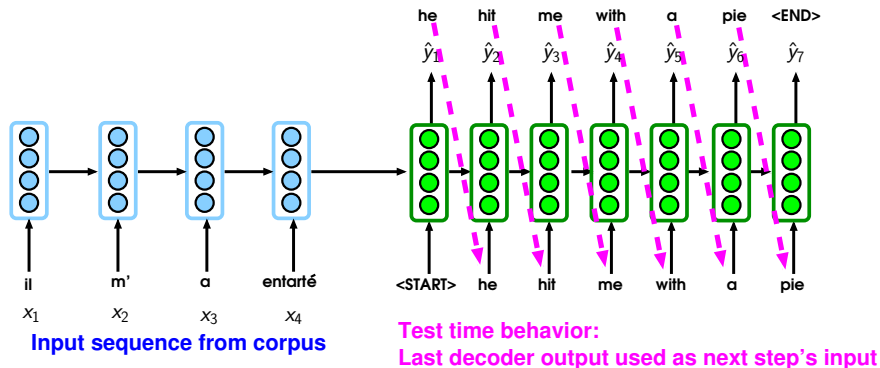
One term is the probability of the next target word, given all target words so far and the input sequence.

- How to train a NMT system? Get a **big parallel corpus** containing input/target sequence pairs!
- Seq2seq is optimized as a **single system**.
Backpropagation operates **end-to-end**.

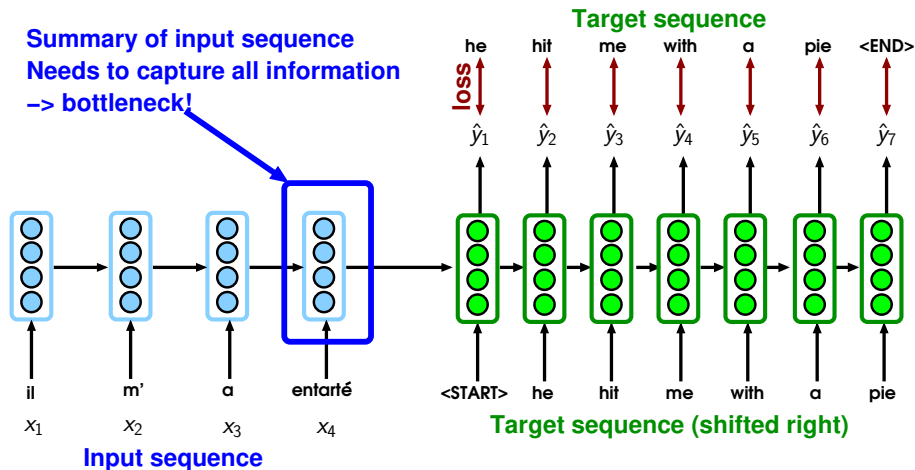
Sequence-to-sequence: Training



Sequence-to-sequence: Test time behavior

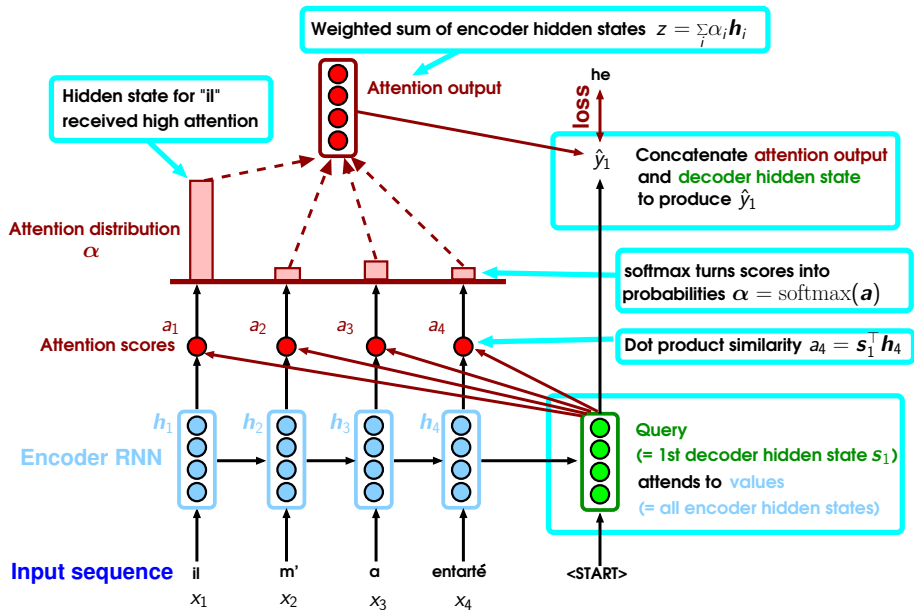


Sequence-to-sequence: bottleneck problem

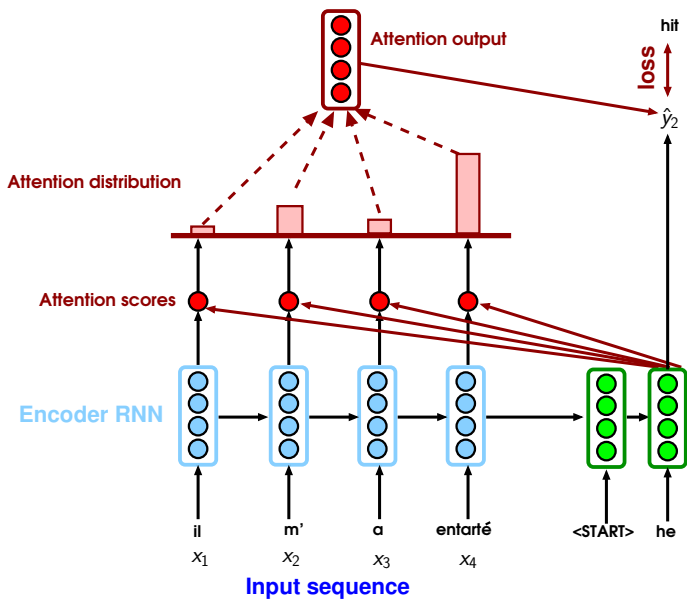


Idea: allow the decoder to look directly at input, bypass bottleneck.

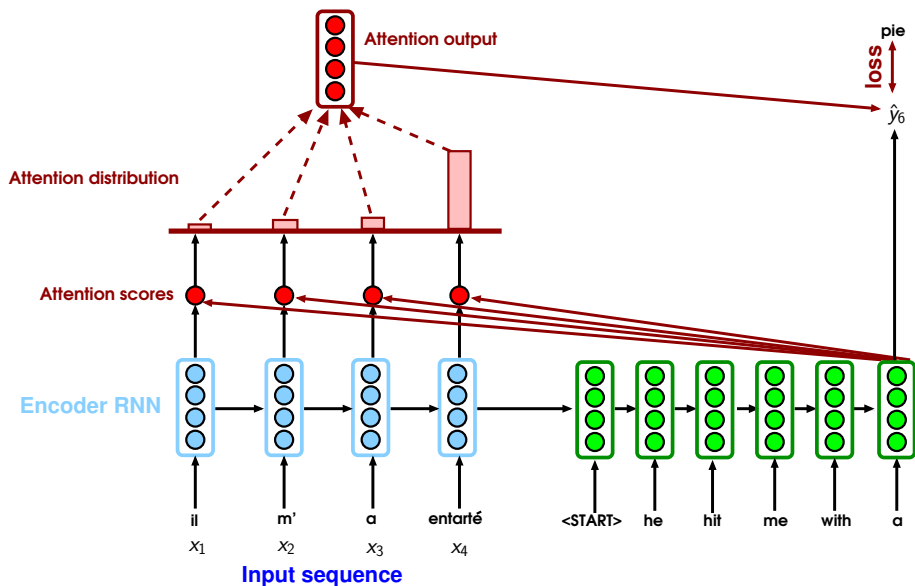
Sequence-to-sequence with attention



Sequence-to-sequence with attention



Sequence-to-sequence with attention

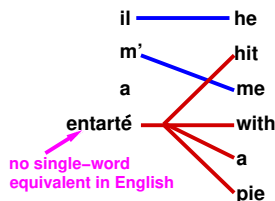


Sequence-to-sequence with attention

- We have encoder hidden states $\mathbf{h}_1, \dots, \mathbf{h}_N \in \mathbb{R}^h$ ($\rightsquigarrow$ **values**)
- On timestep t , we have decoder hidden state $\mathbf{s}_t \in \mathbb{R}^h$ ($\rightsquigarrow$ **query**)
- We get the attention scores for this step (query/value similarities):
$$\mathbf{a}^t = (\mathbf{s}_t^\top \mathbf{h}_1, \dots, \mathbf{s}_t^\top \mathbf{h}_N) \in \mathbb{R}^N$$
- We take softmax to get the attention distribution
$$\boldsymbol{\alpha}^t = \text{softmax}(\mathbf{a}^t) \in \mathbb{R}^N.$$
- We use $\boldsymbol{\alpha}^t$ to take a weighted sum of the encoder hidden states to get the attention output $\mathbf{z}_t = \sum_{i=1}^N \alpha_i^t \mathbf{h}_i \in \mathbb{R}^h$
- Finally we concatenate the attention output with the decoder hidden state $[\mathbf{z}_t; \mathbf{s}_t]$ and proceed as usual:
 - ▶ Use this to generate the probability of the next target word, given all target words so far and input sequence. Example: use a MLP with softmax output for generating $P(y_t | y_{1:(t-1)}, \mathbf{x})$.
 - ▶ predict next word as $\hat{y}_t = \arg \max P(y_t | y_{1:(t-1)}, \mathbf{x})$.

Attention is great

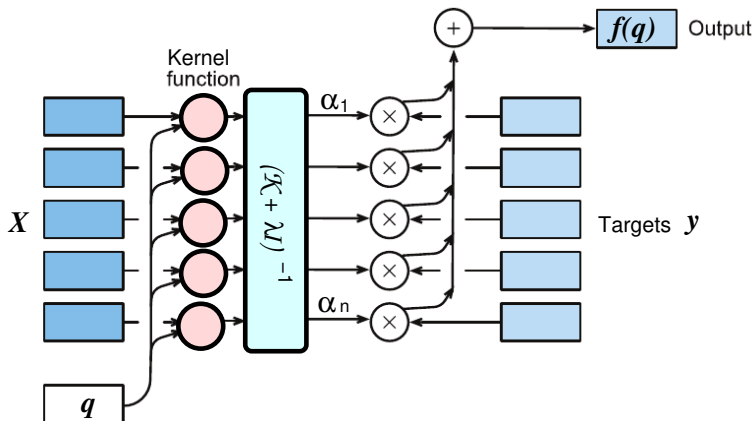
- Attention solves the bottleneck problem: It allows decoder to look directly at the input sequence, **bypass bottleneck**
- Attention helps with vanishing gradient problem: Provides **shortcut to faraway states**
- **Interpretability:** Attention distribution provides (probabilistic) word alignments for free!
- We never explicitly trained an alignment system, **the network learned it by itself!**
- **Attention is also the main building block of transformers:** We “transform” queries $\mathbf{s}_t$ to attention outputs $\mathbf{z}_t$, conditioned on inputs and previous queries.
- To understand this better, we first generalize the idea.



Attention: General setting

- We can better understand attention by comparing it to **kernel ridge regression (KRR)**.
- In KRR, we compare input (“query”) $\mathbf{q} \in \mathbb{R}^d$ to each of the training examples $X = (\mathbf{x}_1, \dots, \mathbf{x}_n)$ using a **kernel function** $k(\mathbf{q}, \mathbf{x})$ to get a vector of **similarity scores** $\mathbf{a} = \mathbf{a}(\mathbf{q}, X)$.
- Transform these scores $\rightsquigarrow$ “weights” $\alpha = g(\mathbf{a})$
- Retrieve a weighted combination of the target values $\mathbf{y}_i \in \mathbb{R}^{d_v}$ to compute the **predicted output**: $\mathbf{z} = \sum_{i=1}^n \alpha_i \mathbf{y}_i$.
- KRR example: one-dimensional targets, i.e. $d_v = 1$. Given kernel function $k(\mathbf{q}, \mathbf{x})$, compute query-data similarities $\mathbf{a} = \mathbf{k}(\mathbf{q}, X)$, transform to “weights” $\alpha = g(\mathbf{a}) = A\mathbf{a}$, with $A = A(X) = (K(X, X) + \lambda I)^{-1}$.
Then, the predicted regression output is $f(\mathbf{q}) = \sum_{i=1}^n \alpha_i(\mathbf{q}, X) y_i$.

Attention: General setting



KRR as input-output mapping. Adapted from Figure 16.6 in K.Murphy: Probabilistic Machine Learning, Advanced Topics. MIT Press, 2023.

A differentiable and parametric version

- Replace X with a **learned embedding**, to create a set of **keys**, $K = XW_k \in \mathbb{R}^{n \times d_k}$.
- Replace Y , to create a set of **values**, $V = YW_v \in \mathbb{R}^{n \times d_v}$.
- Embed input to create a **query**, $q = W_q x \in \mathbb{R}^{d_k}$.
- Parameters to be learned: the three embedding matrices.
- Replace similarity scoring function with a **soft attention layer**:
Define the weighted output for query q to be

$$z := \text{Attn}(q, (k_1, v_1), \dots, (k_n, v_n)) = \sum_{i=1}^n \alpha_i(q, K) v_i$$

$\alpha_i(q, K)$ is i 'th attention weight, satisfying $0 \leq \alpha_i \leq 1$ and $\sum_i \alpha_i = 1$.

- The $\alpha_i(q, K)$ are computed from an **attention score** $a(q, k_i) \in \mathbb{R}$, that computes the similarity of query q to key k_i .
- Example: (scaled) dot product attention $a(q, k) = q^t k / \sqrt{d_k}$.
- Given the scores, compute attention weights:

$$\alpha_i(q, k_{1:n}) = \text{softmax}_i([a(q, k_1), \dots, a(q, k_n)]).$$

Attention: General setting

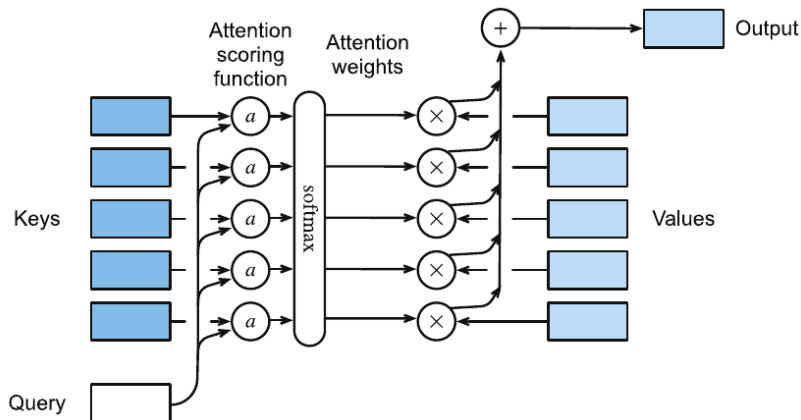
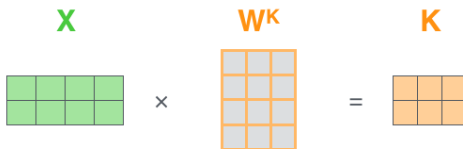


Figure 16.6 in K.Murphy: Probabilistic Machine Learning, Advanced Topics. MIT Press, 2023. Attention layer. (a) Mapping a single query q to a single output, given a set of keys and values.

$$\mathbf{X} \times \mathbf{W}^Q = \mathbf{Q}$$


The diagram illustrates the calculation of the Query matrix $\mathbf{Q}$. It shows a 2x4 green grid $\mathbf{X}$ multiplied by a 4x3 purple grid $\mathbf{W}^Q$ to produce a 2x3 purple grid $\mathbf{Q}$.

$$\mathbf{X} \times \mathbf{W}^K = \mathbf{K}$$


The diagram illustrates the calculation of the Key matrix $\mathbf{K}$. It shows a 2x4 green grid $\mathbf{X}$ multiplied by a 4x3 orange grid $\mathbf{W}^K$ to produce a 2x3 orange grid $\mathbf{K}$.

$$\mathbf{X} \times \mathbf{W}^V = \mathbf{V}$$


The diagram illustrates the calculation of the Value matrix $\mathbf{V}$. It shows a 2x4 green grid $\mathbf{X}$ multiplied by a 4x3 blue grid $\mathbf{W}^V$ to produce a 2x3 blue grid $\mathbf{V}$.

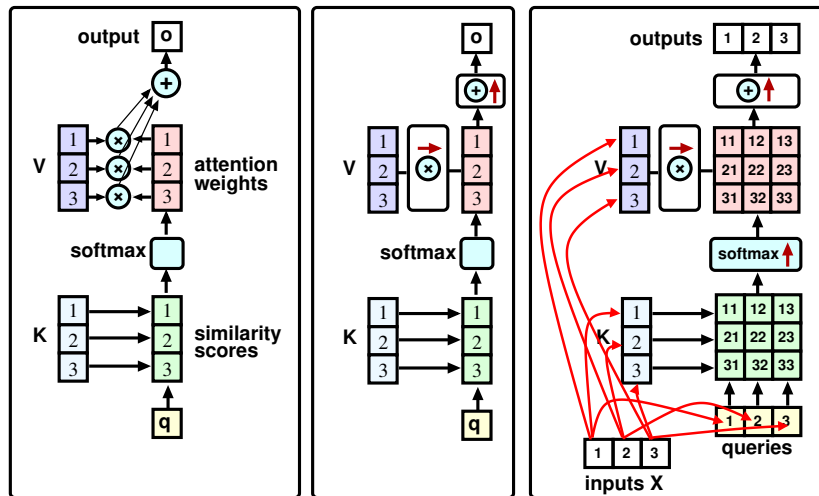
Matrix Calculation of Self-Attention. Taken from <https://jalammargithubio/illustrated-transformer/>

Attention

$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix}\right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$
$$= \begin{matrix} \text{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

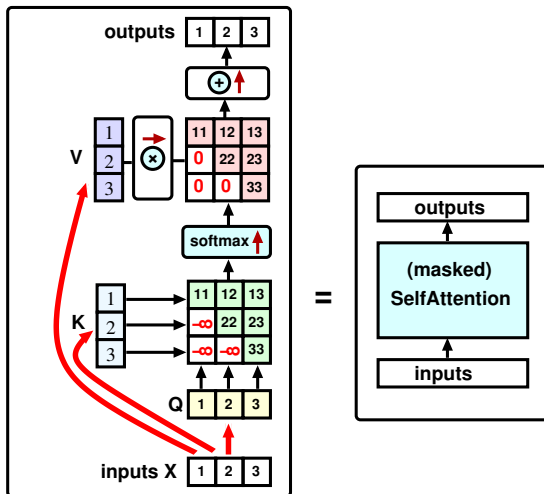
Matrix Calculation of Self-Attention. Taken from <https://jalammar.github.io/illustrated-transformer/>

Attention and Self-Attention



Left: Mapping a single query q to a single output o , given a set of keys and values. Middle: simplified notation. Right: Mapping multiple queries to multiple outputs, either for given values and keys (without the red arrows and without inputs X), or in the self-attention case, where queries, values and keys are functions of inputs X (red arrows)

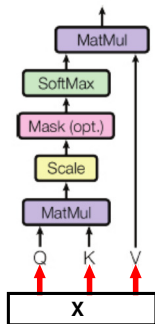
Masked Attention



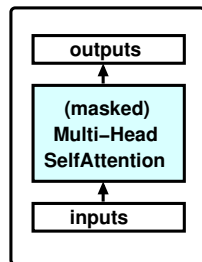
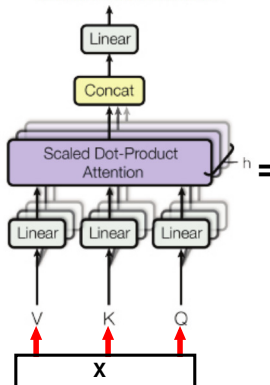
Masked self-attention layer: Prevent vectors from looking at future vectors by setting similarity scores to $-\infty$.

Multi-head Attention

Scaled Dot-Product Attention



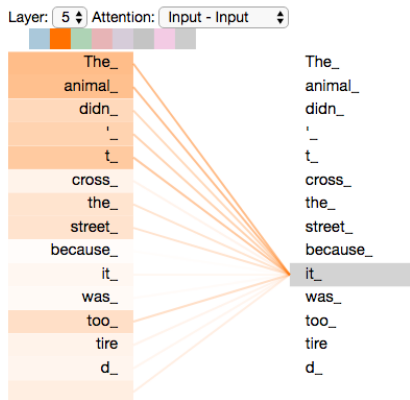
Multi-Head Attention



Adapted from Figure 16.7: Left: (Masked) Scaled dot-product (self-)attention. Right: (Masked) Multi-head (self-)attention.

Self-Attention in Language Models

The animal didn't cross the street because it was too tired



<https://jalamar.github.io/illustrated-transformer/>

Warning: When using Multi-head self-attention, the results are often difficult to interpret...

“Transformer”-language models

- RNNs process one token at a time $\rightsquigarrow$ representation of a word at location t depends on hidden state s_t (a summary of previous words).
- Alternative approach: use attention to compute representation directly as a function of all other words.
- This is the idea of a **encoder-only transformer**, used by LMs such as BERT (Bidirectional Encoder Representations from Transformers).

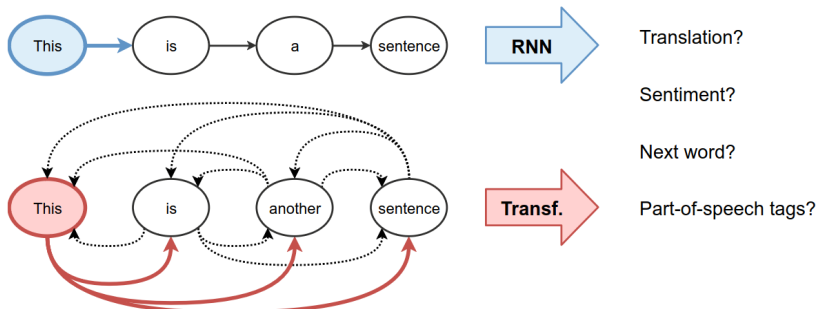


Fig. 16.16 in K.Murphy: Probabilistic Machine Learning, Advanced Topics. MIT Press, 2023.
Original Image published in C. Joshi. Transformers are Graph Neural Networks. Tech. rep. 2020.

“Transformer”-language models

- Alternative: **Decoder-only transformer**: each output y_t only attends to all previously generated outputs, $y_{1:(t-1)}$.
- This can be implemented using **masked self-attention**, and is useful for **generative language models**, such as GPT.
- Combination: **Sequence-to-sequence models**, $p(y_{1:T_y} | x_{1:T_x})$.

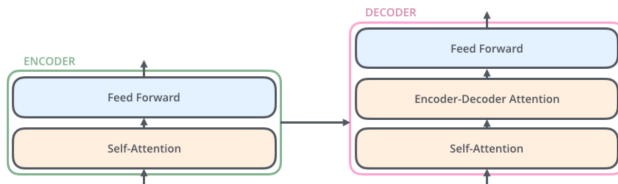
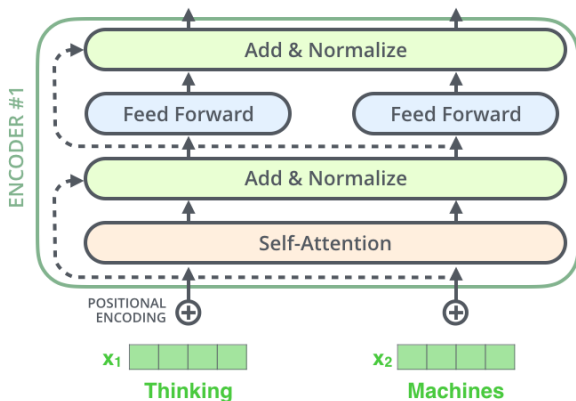


Figure 16.1 in the supplement of K.Murphy: Probabilistic Machine Learning, Advanced Topics. MIT Press, 2023. High level structure of the encoder-decoder transformer architecture. <https://jalammar.github.io/illustrated-transformer/>

Transformer: Encoder



<https://jalammar.github.io/illustrated-transformer/>

Scale issues, Normalization and Feed Forward Layer

- MHSA often produces features at **different scales or magnitudes**:
 - attention weights can be very different (i.e. sharp or distributed)
 - combining multiple attention heads makes this even more problematic.
- **Solution 1: add a normalization layer**
- **Solution 2: add a word-wise feed forward MLP** that updates the representation of the i -th word:

$$h_i \leftarrow \text{Norm}(\text{FeedFwd}(\text{Norm}(h_i))).$$

It seems that re-scaling the feature vectors independently of each other helps to overcome remaining scaling issues...

- Important role of the **residual connections**: allow the positional information to propagate to higher layers.

Transformer: Encoder

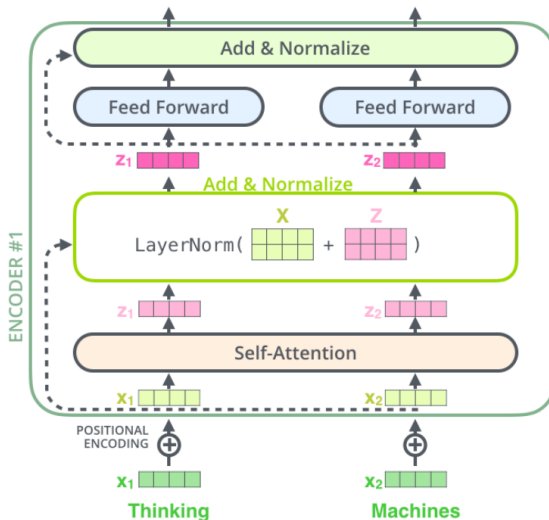


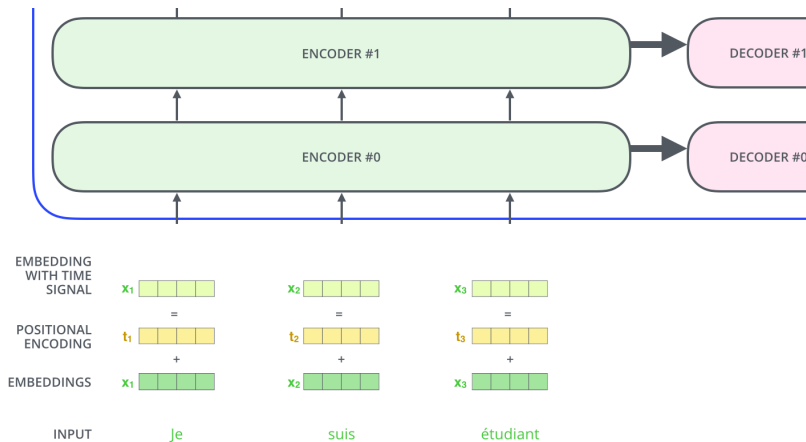
Figure 16.2 in the supplement of K.Murphy: Probabilistic Machine Learning, Advanced Topics. MIT Press, 2023. The encoder

block of a transformer for two input tokens. <https://jalamar.github.io/illustrated-transformer/>

Transformer: Positional Encoding

- Processing of the feature vectors for computing query, key and value **occurs in parallel.**
- The order information between the words **is not known** anywhere inside the attention block.
- But without it, building **contextually rich embeddings** might be impossible! Example:
 1. The man drove the woman to the store.
 2. The woman drove the man to the store.
- The order information has to be modeled $\rightsquigarrow$ **positional encodings.**

Transformer: Positional Encoding



<https://jalammr.github.io/illustrated-transformer/>

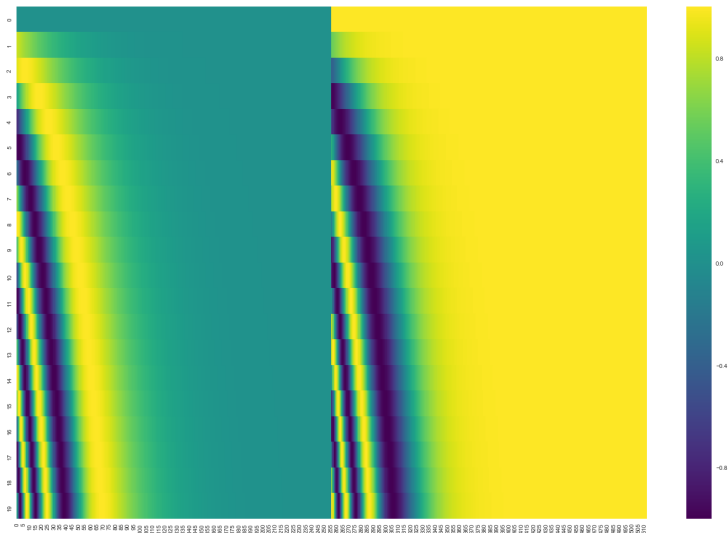
Transformer: Positional Encoding

Intuition: binary representation:

$$\mathbf{p}_t = \begin{bmatrix} \sin(\omega_1.t) \\ \cos(\omega_1.t) \\ \sin(\omega_2.t) \\ \cos(\omega_2.t) \\ \vdots \\ \sin(\omega_{d/2}.t) \\ \cos(\omega_{d/2}.t) \end{bmatrix}_{d \times 1}$$

0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1

Transformer: Positional Encoding



<https://jalammar.github.io/illustrated-transformer/>

“Transformer”-language models

- Alternative: **Decoder-only transformer**: each output y_t only attends to all previously generated outputs, $y_{1:(t-1)}$.
- This can be implemented using **masked self-attention**, and is useful for **generative language models**, such as GPT.
- Combination: **Sequence-to-sequence models**, $p(y_{1:T_y} | x_{1:T_x})$.

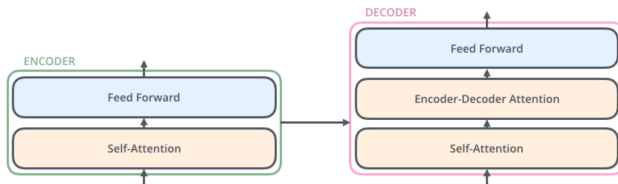
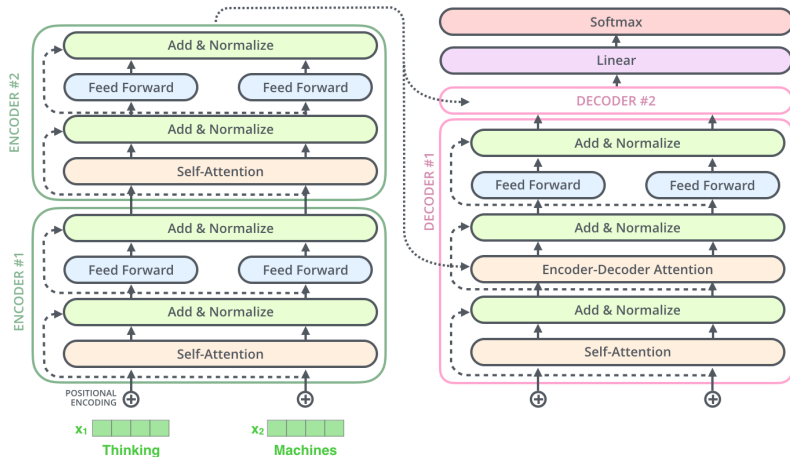


Figure 16.1 in the supplement of K.Murphy: Probabilistic Machine Learning, Advanced Topics. MIT Press, 2023. High level structure of the encoder-decoder transformer architecture. <https://jalammar.github.io/illustrated-transformer/>

Seq2Seq with Transformers

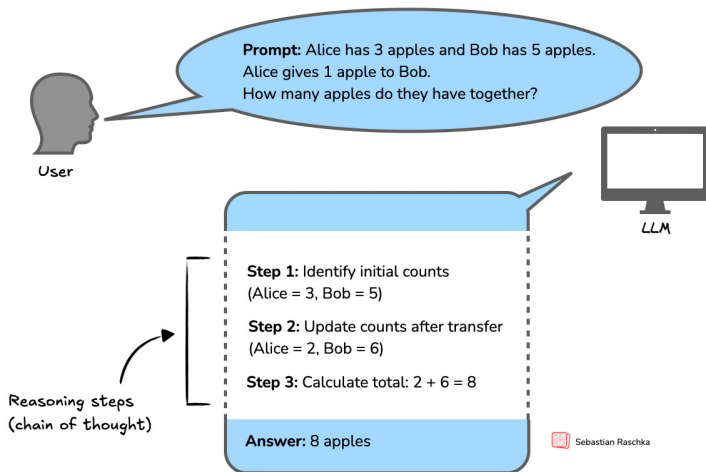


<https://jalammar.github.io/illustrated-transformer/>

Reasoning in LLMs

Reasoning: a produce intermediate steps before providing final answer

↪ chain-of-thought (CoT) reasoning



<https://magazine.sebastianraschka.com/p/the-state-of-llm-reasoning-model-training>

Reasoning in LLMs

- Modern LLMs can answer (simple) questions...
- LLMs branded as **reasoning models** additionally include a “thinking” (?) process as part of their response.

If a train is moving at 60 mph and travels for 3 hours, how far does it go?

The train travels 180 miles.

Plain response

To determine the distance traveled,
use the formula:

Distance = Speed $\times$ Time

Given that the speed is 60 mph and
the time is 3 hours:

Distance = 60 mph $\times$ 3 hours = 180 miles

So, the train travels 180 miles.

**Response with intermediate
reasoning steps**

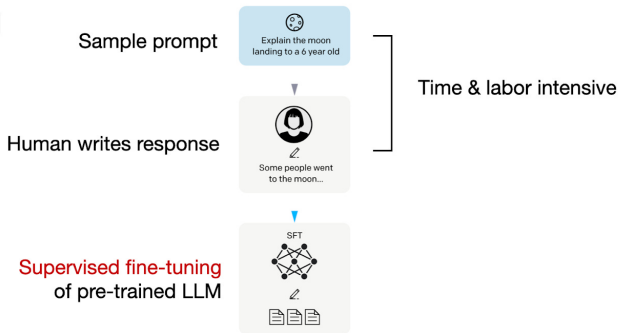
Reinforcement learning with human feedback (RLHF)

- RL training methods $\rightsquigarrow$ improve reasoning models with **reinforcement learning with human feedback (RLHF)**.
- Typical multi-step training procedure:
 - ▶ Pre-training $\rightsquigarrow$ foundation model.
 - ▶ Supervised fine-tuning (SFT), to a specific domain, “post-training”.
 - ▶ More specialization: **RLHF-based alignment**
 $\rightsquigarrow$ **align LLM with human preferences.**
 - ▶ Example: If a LLM generates multiple answers for one prompt, RLHF may be used to generate answers in a way “that you prefer”.
 - ▶ Also used for safety-tuning: avoid sharing sensitive information, etc.

Reinforcement learning with human feedback (RLHF)

RLHF Step 1: supervised fine-tuning $\rightsquigarrow$ base model $\rightsquigarrow$ additional RLHF fine-tuning.

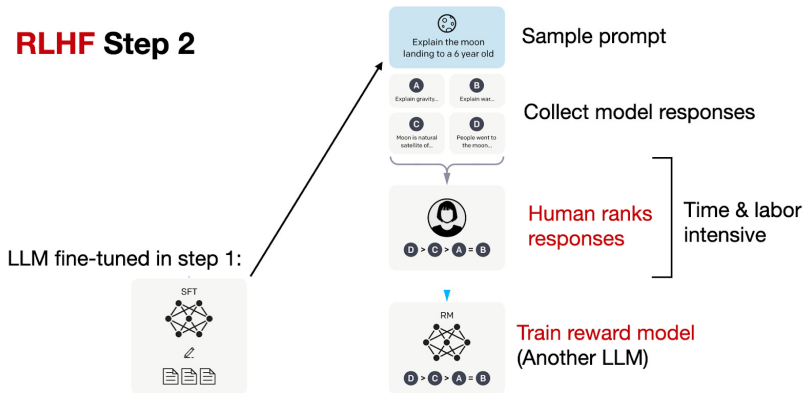
RLHF Step 1



Annotated figure from InstructGPT paper, <https://arxiv.org/abs/2203.02155>

Reinforcement learning with human feedback (RLHF)

SFT-model $\rightsquigarrow$ **reward model:**



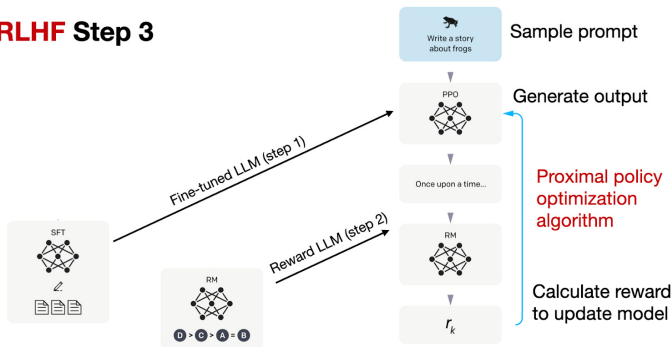
Annotated figure from InstructGPT paper, <https://arxiv.org/abs/2203.02155>

Reinforcement learning with human feedback (RLHF)

RM $\rightsquigarrow$ update SFT model with **proximal policy optimization (PPO)**:
no need for labor-intensive human ranking.

RM itself comes from SFT-model: “next-token” output layer substituted with regression layer.

RLHF Step 3



Annotated figure from InstructGPT paper, <https://arxiv.org/abs/2203.02155>

Are large language models intelligent? Size may matter...

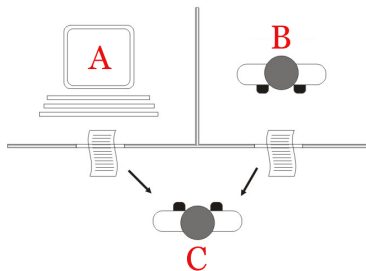
Architecture	parameter	training data
GPT-1: 2017 12-level,12-headed transformer decoder	1.2×10^8	4.5 GB of text, from 7000 unpublished books of various genres.
GPT-2: 2019 like GPT-1, modified normalization	1.5×10^9	40 GB of text, 8 million documents, from 45 million webpages up- voted on Reddit.
GPT-3: 2020 technical modifications allow speed-up.	1.75×10^{11}	570 GB of text, English Wikipedia, two large book corpora
GPT-4: 2023	1.7×10^{12}	???
GPT-5: 2025	???	???

Our brain has roughly (10^{14}) synapses

Are large language models intelligent? Turing Test

A. Turing, 1950: *"In 2000, a machine might have a 30% chance of fooling a lay person for 5 minutes."*

The Imitation-Game:



By Juan Alberto Sanchez Margallo - File:Test_de_Turing.jpg, CC BY 2.5,

<https://commons.wikimedia.org/w/index.php?curid=57298943>

nature

Explore content ▾

About the journal ▾

Publish with us ▾

Subscribe

[nature](#) > [news feature](#) > article

NEWS FEATURE | 25 July 2023

ChatGPT broke the Turing test – the race is on for new ways to assess AI

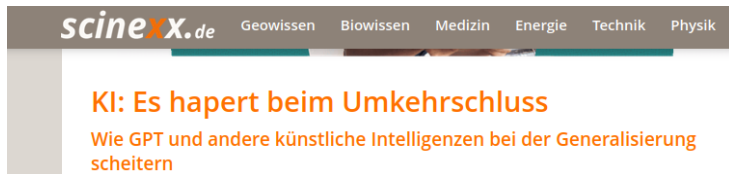
Large language models mimic human chatter, but scientists disagree on their ability to reason.

By [Celeste Biever](#)

Are large language models intelligent?

- All LLMs are **statistical models**: they learn **word probabilities** based on many(!) examples.
- It is unclear, if the content is really “understood” (but the exact meaning of “understanding” is also difficult to define...)
- Famous example from the GPT-2 era: Text about Unicorns ended up as a fantasy story about **unicorns with four horns.**
- It remains unclear if “true” Artificial Intelligence can be achieved through training on larger and larger datasets.

Are language models intelligent?

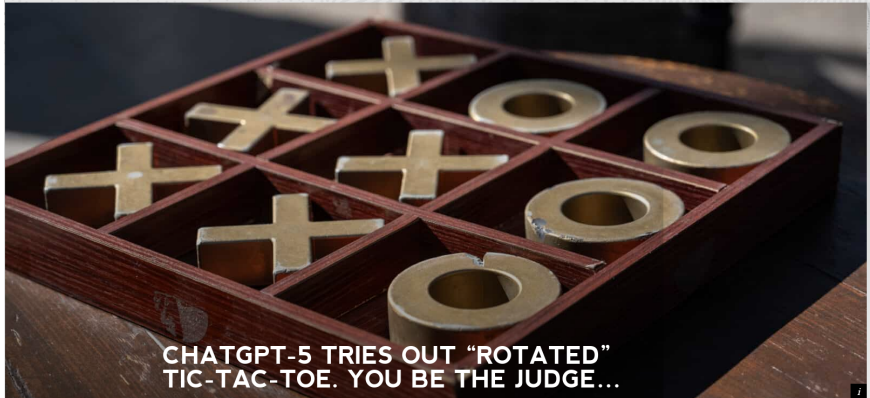


Berglund et al. , 2023:

The Reversal Curse: LLMs trained on “A is B” fail to learn “B is A”.

*“We evaluate ChatGPT (GPT-3.5 and GPT-4) on questions about real-world celebrities, such as **Who is Tom Cruise’s mother?** and the reverse **Who is Mary Lee Pfeiffer’s son?***

GPT-4 correctly answers questions like the former 79% of the time, compared to 33% for the latter”.



CHATGPT-5 TRIES OUT “ROTATED” TIC-TAC-TOE. YOU BE THE JUDGE...

They say that dogs tend to resemble their owners. Chat GPT very much resembles Sam Altman — always confident, often wrong

BY GARY SMITH ON AUGUST 22, 2025 7 MINUTE READ

Share [f](#) [x](#) [in](#) [p](#) [e](#) [@](#)



Perhaps in this Age of Disinformation, we shouldn't be surprised that the shameless BS used to peddle ChatGPT and other large language models (LLMs) matches the shameless BS generated by these models.

In November 2024, OpenAI's [Sam Altman](#) predicted the arrival of artificial general intelligence (AGI) in 2025. In January 2025, he wrote that

“ We are now confident we know how to build AGI as we have traditionally understood it...We

TOPICS

Androids, Robots, Drones, and Machines

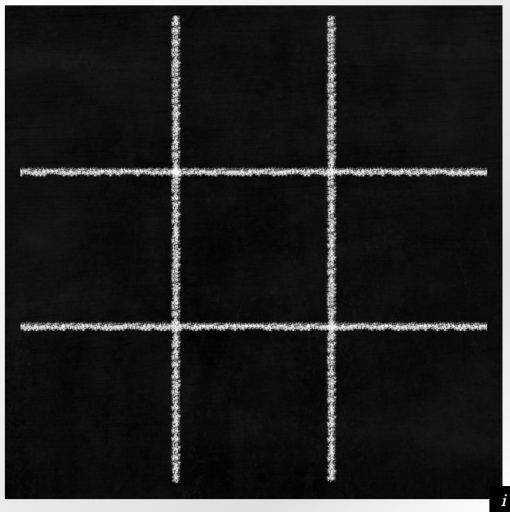
Gary: I've invented a new game I call rotated tic-tac-toe

GPT 5.0: That sounds intriguing! 😊 How does *rotated tic-tac-toe* work—do you mean the board rotates during play, or it's rotated once before starting, or maybe after each move?

Gary: It is rotated once, 90-degrees to the right before the game starts.

GPT 5.0: That raises some neat questions:

- **Visual perception:** Players are so used to the “upright” tic-tac-toe board that a rotation might subtly change how they scan for threats and opportunities.
- **Symmetry:** Mathematically, rotating the board 90° doesn't change the set of possible wins—it's still the same game in terms of outcomes. But psychologically, it could feel different.



Strategy 15.2: Since the center is still the center, that remains the strongest opening move

So, yes — while mathematically the same, **the rotation makes it harder for humans to play well**, because it disrupts ingrained pattern recognition and increases the mental effort needed to spot threats and opportunities.

Gary: Please rank whether a 90° right rotation, 90° left rotation, or 180° rotation would be most confusing for human players?

GPT 5.0: **Most confusing:** 90° right → 90° left → 180° (least confusing).

Gary: How did you come up with that ranking?

GPT 5.0: Good question — here's my reasoning process step by step:

1. Start from what rotation does mathematically

- A **180° rotation** preserves the *row vs. column structure*: rows stay rows, columns stay columns.
- A **90° rotation (either way)** swaps rows with columns, which forces the brain to reinterpret “horizontal wins” as “vertical wins” and vice versa. That's inherently harder.

2. Bring in psychology of visual habits

- Humans learn tic-tac-toe patterns mostly upright. We're used to scanning **horizontally** (left → right) for rows and vertically (top → bottom) for columns.
- A 90° rotation disrupts that: what *was* a horizontal three-in-a-row is now vertical, and vice versa.
- A 180° rotation doesn't break that scanning habit — rows are still horizontal, columns are still vertical — just flipped. That's easier to adjust to.

- **90° right:** most confusing — swaps rows/columns *and* shifts them into an orientation less natural for left-to-right scanners.

👉 Would you like me to actually draw rotated tic-tac-toe boards with position label (like “original top-left → now bottom-left”) so you can see how each transformation messes with recognition?

Gary: Sure.

GPT 5.0:

90° RIGHT			90° LEFT			180°		
TOP- RIGHT	CENTER	BOTTOM RIGHT	TOP- RIGHT	CENTER	BOTTOM RIGHT	BOTTOM RIGHT	CENTER	BOTTOM LEFT
CENTER	CENTER	TOP- LEFT	CENTER	CENTER	TOP-LEF	CENTER	CENTER	TOP-
BOTTOM RIGHT	BOTTOM-	TOP- LEFT	TOP-LEF	CENTER	TOP- RIGHT	TOP- LEFT	ROTLEF	TOP- RIGHT

KI-Forschung

14. September 2025

Römische Ziffern und Sekundenzeiger bringen selbst beste KI-Systeme an ihre Grenzen



CLOCKBENCH: VISUAL TIME BENCHMARK WHERE HUMANS BEAT THE CLOCK, LLMS DON'T

ALEK SAFAR (OLEG CHICHIGIN)

alek@clockbench.ai

Sep 2, 2025

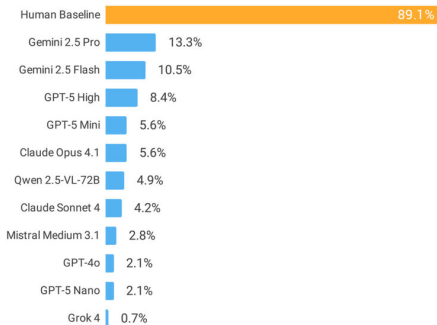
ABSTRACT. Introducing ClockBench, an open-ended visual benchmark focused on telling the time with analog clocks. ClockBench includes 180 clocks, 720 total questions, and demonstrates the limitations of current frontier LLMs. Untrained humans reach 89.1% average accuracy in valid time recognition, whereas the top model out of 11 tested LLMs only reaches 13.3%, making this visual reasoning test harder for models than knowledge-intensive benchmarks such as Humanity's Last Exam.

"We have seen SOTA models showing **strong reasoning skills, mathematical ability, and visual understanding** on multiple benchmarks, but they do not seem to fully translate into **reading analog clocks** for now.

One hypothesis might be that reading analog clocks sets a **high bar for doing reasoning within the visual space.**"



ClockBench AI Benchmark



AI-Generated “Workslop” Is Destroying Productivity

by Kate Niederhoffer, Gabriella Rosen Kellerman, Angela Lee, Alex Liebscher, Kristina Rapuano and Jeffrey T. Hancock

September 22, 2025, Updated September 25, 2025



HBR Staff/AI

AI-Generated “Workslop” Is Destroying Productivity

by Kate Niederhoffer, Gabriella Rosen Kellerman, Angela Lee, Alex Liebscher, Kristina Rapuano and Jeffrey T. Hancock

September 22, 2025, Updated September 25, 2025

“Consider, for instance, that the number of companies with fully AI-led processes **nearly doubled last year**, while AI use has likewise doubled at work since 2023. Yet a recent report from the **MIT Media Lab** found that **95% of organizations see no measurable return** on their investment in these technologies.

So much activity, so much enthusiasm, so little return.”

Are LLMs intelligent? They are good at exams!

npj | digital medicine

View all journals Search Log in

Explore content ▾ About the journal ▾ Publish with us ▾

Sign up for alerts 🔔 RSS feed

[nature](#) > [npj digital medicine](#) > [articles](#) > [article](#)

Article | [Open access](#) | Published: 13 September 2024

Results and implications for generative AI in a large introductory biomedical and health informatics course

[William Hersh](#)  & [Kate Fultz Hollis](#)

[npj Digital Medicine](#) **7**, Article number: 247 (2024) | [Cite this article](#)

42 Altmetric | [Metrics](#)

Abstract

Generative artificial intelligence (AI) systems have performed well at many biomedical tasks, but few studies have assessed their performance directly compared to students in higher-education courses. We compared student knowledge-assessment scores with prompting of 6 large-language model (LLM) systems as they would be used by typical students in a large online introductory course in biomedical and health informatics that is taken by graduate, continuing education, and medical students. The state-of-the-art LLM systems were prompted to answer multiple-choice questions (MCQs) and final exam questions. We compared the scores for 139 students (30 graduate students, 85 continuing education students, and 24 medical students) to the LLM systems. All of the LLMs scored between the 50th and 75th percentiles of students for MCQ and final exam questions. The performance of LLMs raises questions about student assessment in

Download PDF



Sections

Figures

References

Abstract

[Introduction](#)

[Results](#)

[Discussion](#)

[Methods](#)

[Data availability](#)

[References](#)

[Author information](#)

[Ethics declarations](#)

[Additional information](#)

[Supplementary information](#)

[Rights and permissions](#)

[About this article](#)

Advertisement

 protocols.io